COMP9334 Project:

Priority queueing for server farms with batch arrivals

Q1(a)

This was tested on the CSE lab machine, and I passed all the provided test cases from Test 1 to Test 7. Please refer to the screenshot below showing the output and validation of these tests.

```
=== Running test 1... ===
MRT = 4.8143
Results saved to output/mrt_1.txt and output/dep_1.txt

Test 1: Mean response time matches the reference
Test 1: Departure times match the reference

=== Running test 2... ===
MRT = 5.0571
Results saved to output/mrt_2.txt and output/dep_2.txt

Test 2: Mean response time matches the reference
Test 2: Departure times match the reference

=== Running test 3... ===
MRT = 1.9101
Results saved to output/mrt_3.txt and output/dep_3.txt

Test 3: Mean response time matches the reference
Test 3: Departure times match the reference

=== Running test 4... ===
MRT = 2.4023
Results saved to output/mrt_4.txt and output/dep_4.txt

Test 4: Mean response time matches the reference
Test 4: Departure times match the reference
```

```
=== Running test 5... ===
MRT = 1.5897
Results saved to output/mrt_5.txt and output/dep_5.txt

Test 5: Mean response time is within tolerance

=== Running test 6... ===
MRT = 1.5213
Results saved to output/mrt_6.txt and output/dep_6.txt

Test 6: Mean response time is within tolerance

=== Running test 7... ===
MRT = 2.4693
Results saved to output/mrt_7.txt and output/dep_7.txt

Test 7: Mean response time is within tolerance
```

Q1(b)

In this problem, we need to check the correctness of the random number generator in random mode to make sure that the arrival intervals, the number of sub-jobs and the service times used in the simulation match the theoretical distribution. To do this, based on the code at week4b and week5, we generated two files for analysis, sub-job.py and servicetime.py. sub-job.py generates 200,000 samples and plots the histograms of the three random variables against the theoretical distribution. And the servicetime.py focuses on service time, generating 100,000 samples and plotting CDF comparisons.

In order to prove that the random number generator we use in random mode correctly simulates the distribution of probability theoretically, we verified it for three random variables:

Inter-arrival time, the number of sub-jobs, and the service time.

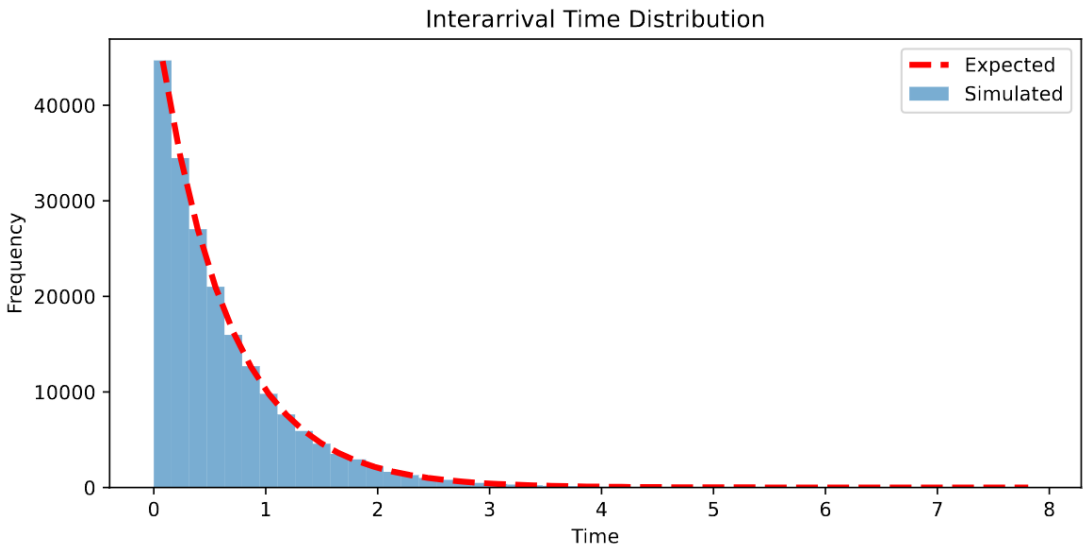
The following graphs are drawn for this purpose:

The inter-arrival probability distribution:

The arrival time is generated using the following equation:

Interarrival=Exponential(λ=1.6)×Uniform(a2l=0.9,a2u=1.1)

The values of a2l and a2u in the code are 0.9 and 1.1, respectively. This is used to simulate the fluctuation of the arrival time. Graph shows the comparison of the modelled arrival interval with the theoretical density curve. Graph shows the comparison between the simulated arrival intervals and the theoretical density curves. It can be seen that the simulated and theoretical indices are in good agreement, showing that the generator can correctly simulate the expected arrival behavior.



Probability distribution of the number of sub-jobs:

To check the correctness of the random number generator's distribution of pairs of jobs in a randomized mode, we took 100,000 samples by using the same probability vectors as in the simulation, via np.random.choice(). The experiential frequencies are then compared to the theoretical distribution.

The probabilities of TEST are as follows:

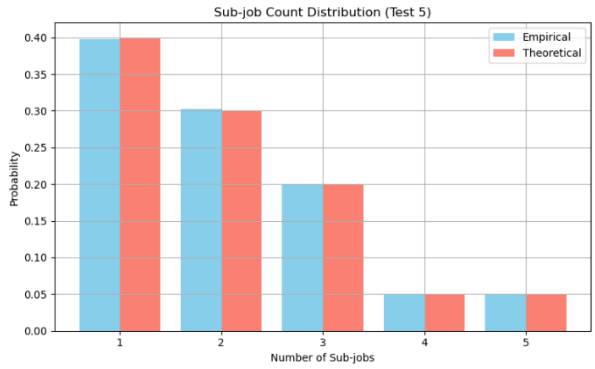
Test5: 0.400 0.300 0.200 0.050 0.050

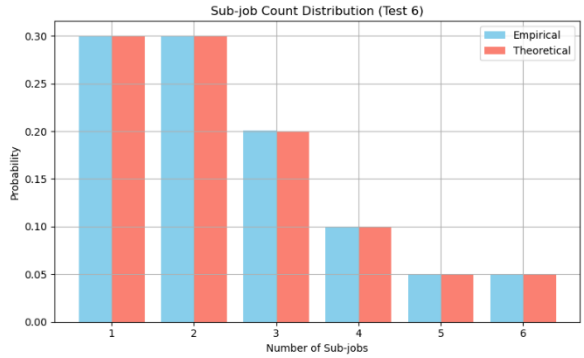
Test6: 0.300 0.300 0.200 0.100 0.050 0.050

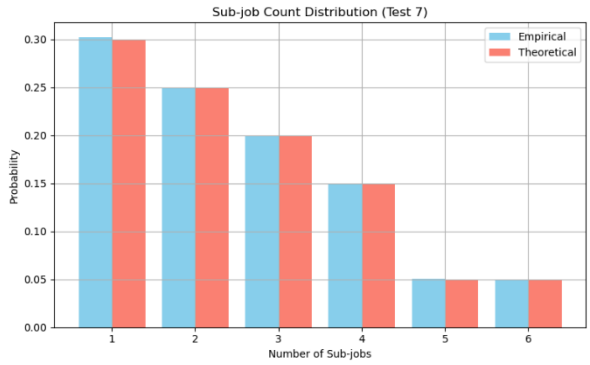
Test7: 0.300 0.250 0.200 0.150 0.050 0.050

As shown in the figure below, the results are the same as the theoretical probabilities , which confirms that the behavior of the sub-job generator is as

expected.



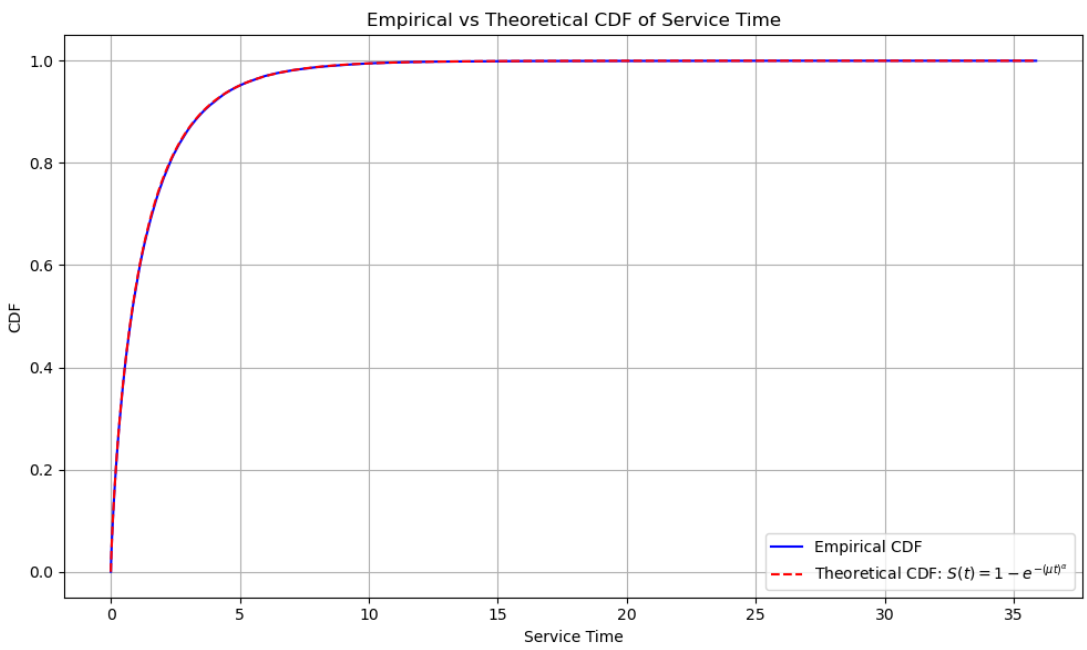




Service time distribution:

For the first graph, we should prove that my code correctly generates random variables that match the distribution or not. I used Inverse Transform Sampling to produce many service time samples and compare them with the theory distribution.

In the graph below, the blue curve is the CDF based on 100,000 simulated samples, and the red dotted line is the theoretical CDF. The two match well, proving that the service time random number generator I implemented correctly reflects the target distribution.



For the second Graph, the service time is generated using the following formula:

CDF formula for the service time of the suboperation provided in the topic:

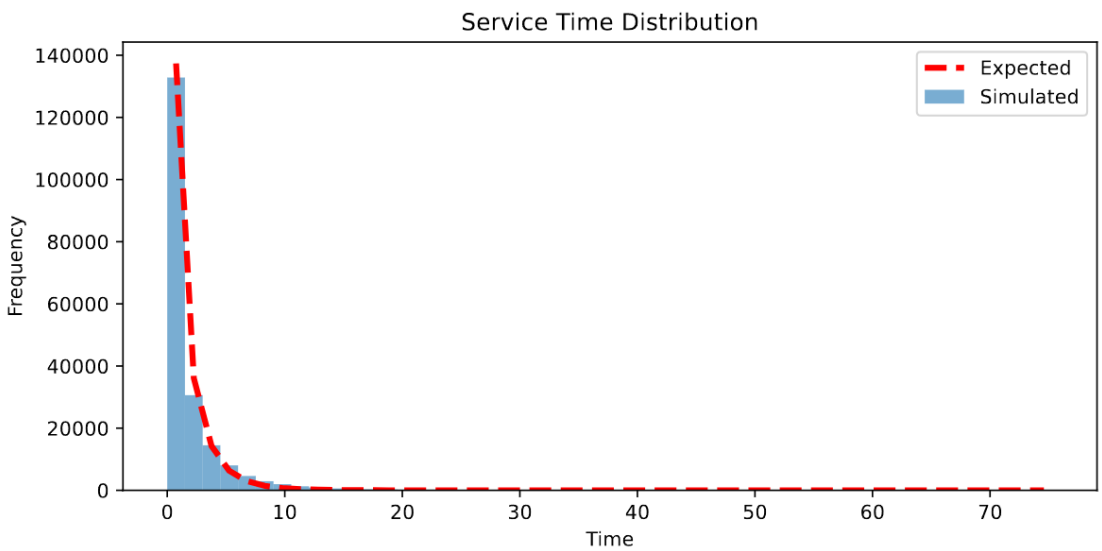
S(t) = 1-exp(-(μt) ^α)

A sampling formula for the service time is obtained using the Inverse Transform Sampling technique:

t= 1/μ ·(-ln(1-u)) ^(1/α)

where U ~ Uniform (0,1), μ=0.8 and α=0.8 in this experiment.

From the graph below, it can be observed that the histograms of the simulated data are in highly match with the theoretical distribution curve, and all of them show the characteristics of right skewed with longer tails, which is in line with the expected shape.



During the verification process, we first used np.random.exponential and np.random.uniform in sub-job.py to generate the inter-arrival time with parameters λ=1.6, a2l=0.9, a2u=1.1, and compared the histogram with the theoretical exponential distribution to confirm its consistency. For the number of sub-jobs, we use np.random.choice to generate samples according to the given probability vector pk=[0.22, 0.28, 0.3, 0.08, 0.07, 0.05] and compare them with the theoretical discrete distribution. The service time adopts the inverse transform sampling method. Its formula is t = (1/μ) * (-ln(1-u))^(1/α), where μ=0.8, α=0.8. After generating the sample, a histogram is plotted to compare with the theoretical Weibull distribution. In addition, in servicetime.py, we further confirm the accuracy of the service time generator through CDF analysis. The high agreement between the simulation data and the theoretical curve S(t)=1-exp(-(μt)^α) indicates that the generator is reliable.

Q2

In this problem, we wanted to control the randomness of the simulation and verify its reproducibility. To do so, we used the file stable_seed.py to run 5 simulations for each seed by setting different random seeds (0 to 39), calculate the MRT, and plot box plots to observe the stability and distribution of the MRT with different seeds.

To control the randomness and ensure the repeatability of the simulation, I used

```
for seed in range(num_replications):
```

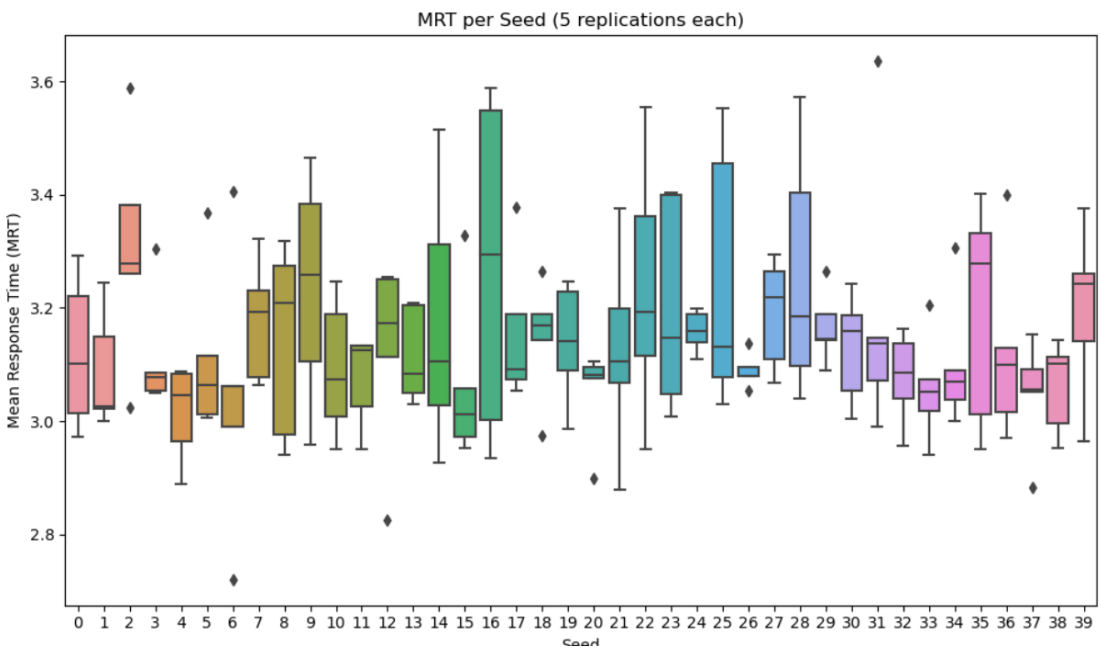
np.random.seed(seed)

in main,py random mode, and used NumPy random functions such as np.random.choice and np.random.exponential to ensure the consistency of the results.

This means that I run the simulation once for each seed, which not only keeps the arrival time, number of child jobs, and service time consistent from one simulation to the another, but also allows me to compare the stability of the MRT in different seeds.

In order to verify the reproducibility of the simulation results, I fix different random seed seed values (0~39), and repeat the simulation 5 times under each seed, calculate the mean response time (MRT) of each simulation, and draw a box-and-whisker plot as shown in the figure.

From the graph, we can observe that even though the random seeds are different, the results of each group still show a consistent trend, and the median MRT distribution is concentrated under most of the seeds, which indicates that the simulation program has good stability and reproducibility.



During the simulation, we set fixed parameters in stable_seed.py (h=3, time_end=2000, λ=1.6, μ=0.8, α=0.8, etc.) and used np.random.seed(seed) to ensure that the random number sequence of each simulation is consistent. For each seed, we generate the arrival time, number of sub-jobs, and service time, run the simulation, and record the MRT. The box plots show that although there are slight variations in MRT between different seeds, the median distribution is concentrated and the box range is relatively consistent, indicating that the simulation results have good stability over multiple runs. This method not only verifies the controllability of random number generation, but also proves the reliability of the simulation system under different initial conditions.

Q3(a)

In Q3(a), we need to determine the appropriate length, number of repetitions, and transient cutoff time of the simulation to ensure that the system enters the stable MRT. We used several files for this analysis:

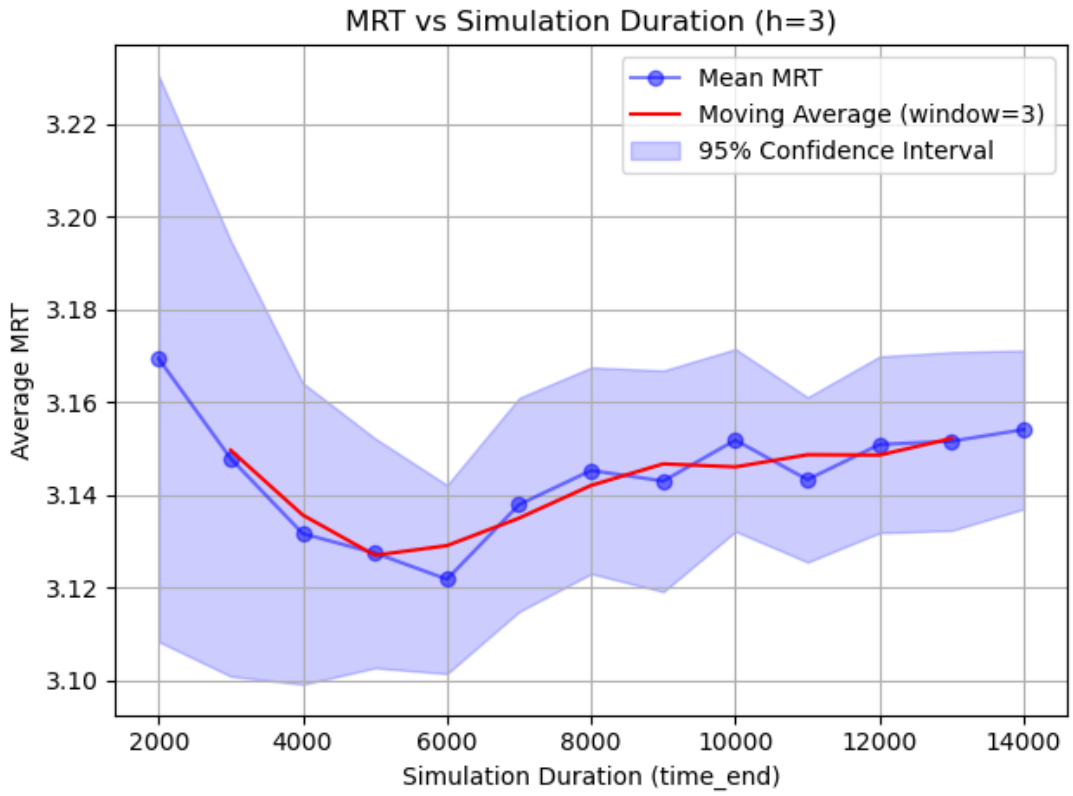
time_end_test.py to analyze the simulation length.

mrt_cutoff_time.py to analyze the cutoff sensitivity;

slidingwindow.py and "cumulative_mean_response_time.png" to observe the steady state trend.

Simulation length:

The following figure shows the trend of the mean response time for h=3 with simulation durations ranging from 2000 to 14000 seconds. In the early stage (2000 to 4000 seconds), the average MRT decreases from around 3.17 to 3.13, indicating a gradual transition from the transient phase toward stability.From 4000 to 6000 seconds, the MRT continues to stabilise, and the width of the 95% confidence interval narrows significantly from approximately 0.10 to 0.04, reflecting increased data stability. After 8000 seconds, the MRT remains stable around 3.15–3.16, and the moving average flattens, confirming steady-state conditions.Based on this trend, it is reasonable to choose time_end = 8000 as it provides a balance between simulation accuracy and computational cost.



Replications:

In this experiment, 5 simulations are performed for each h-value. As shown in the figure below, each seed was simulated 5 times for each of the 40 boxplots with different random seeds. The results showed some fluctuations but the overall trend was stable, indicating that 5 replications were sufficient to observe the system behavior. In addition, I tried to use 10 simulations, which took too long, so I finally chose 5 replications.

Transient cutoff time:

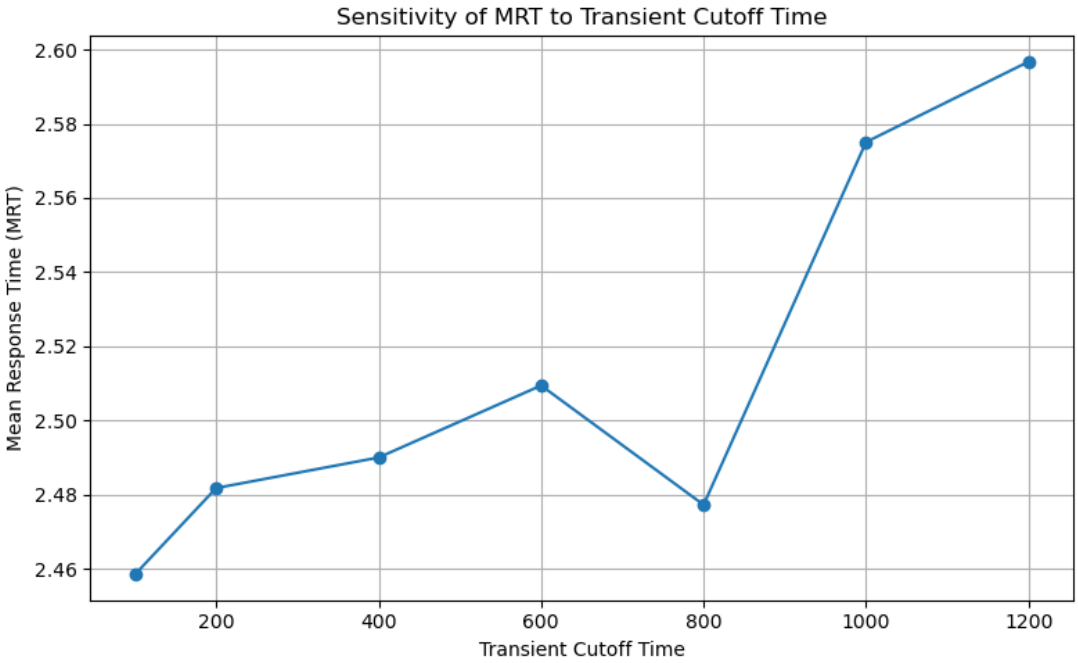
The first graph is the Sensitivity of MRT to Transient Cutoff Time

graph, which shows the trend of the average response time as the transient cutoff time changes.

As the cutoff time increases from 0 to about 600, the MRT rises slowly from 2.46 to 2.51. It then drops to 2.48 at 800.

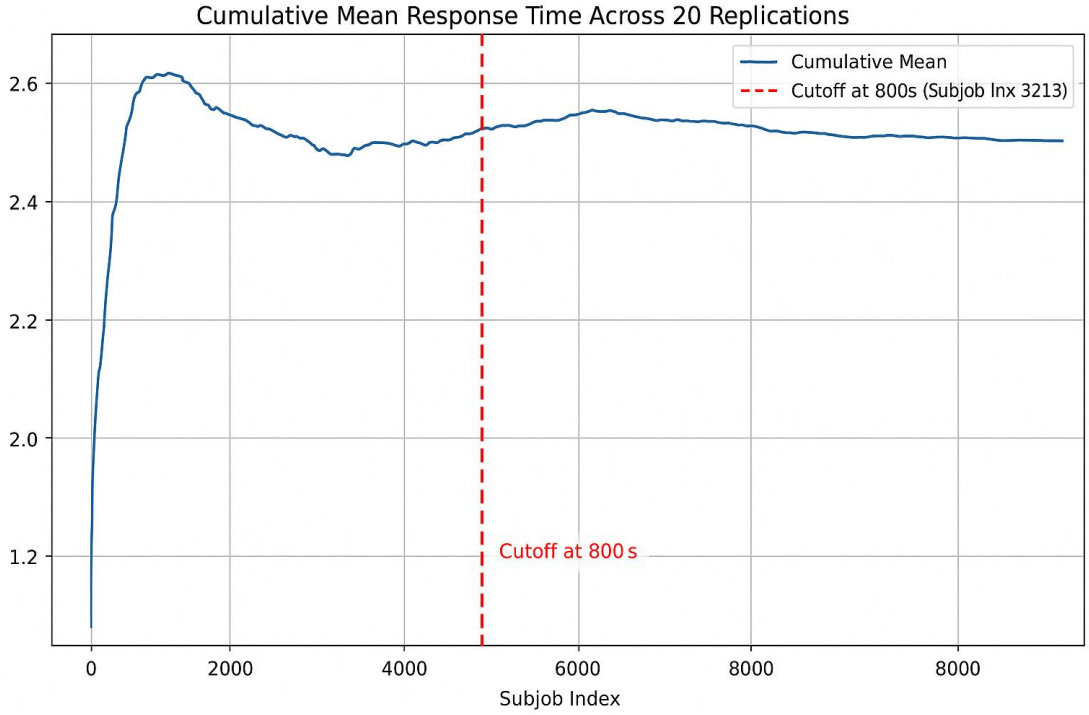
After that, the MRT rises rapidly and reaches 2.60 at the cutoff time of 1200.

From this, it can be seen that setting the cutoff time at 800 may be a suitable choice, because the MRT which is 2.48 reaches a local minimum here, it shows that the transient phase is probably finished and the system is starting to enter a steady state.



The second graph shows the Cumulative Mean Response Time Across 20 Replications, which uses the cumulative average method to observe the data trend.

At Subjob Index 3500 (833 seconds), the curve flattens out completely and the fluctuations almost disappear, indicating that the system has entered a stable state. However, at 800 seconds, the fluctuations have reduced significantly and are beginning to level off.



The third graph shows the Smoothed Mean Response Time Across 20 Replications, which is smoothed by the sliding average method, with smoothing window sizes of W=200, W=400, and W=600.

From the graphs, we can see that the fluctuation is larger at W=200, but it can be seen that the response time fluctuates between 2-3 seconds.

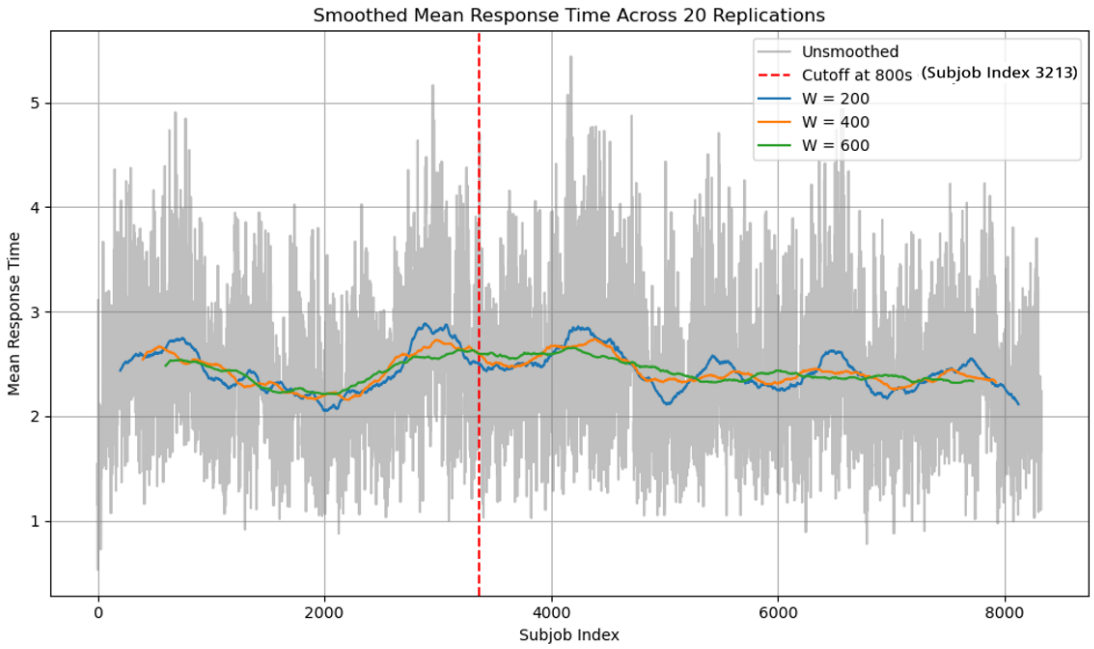
Next, the fluctuation of W=400 is reduced, the curve is smoother and the trend is more obvious.

At W=600 the fluctuation is the smallest; the curve is the smoothest and the response time is stable at around 2.5 seconds.

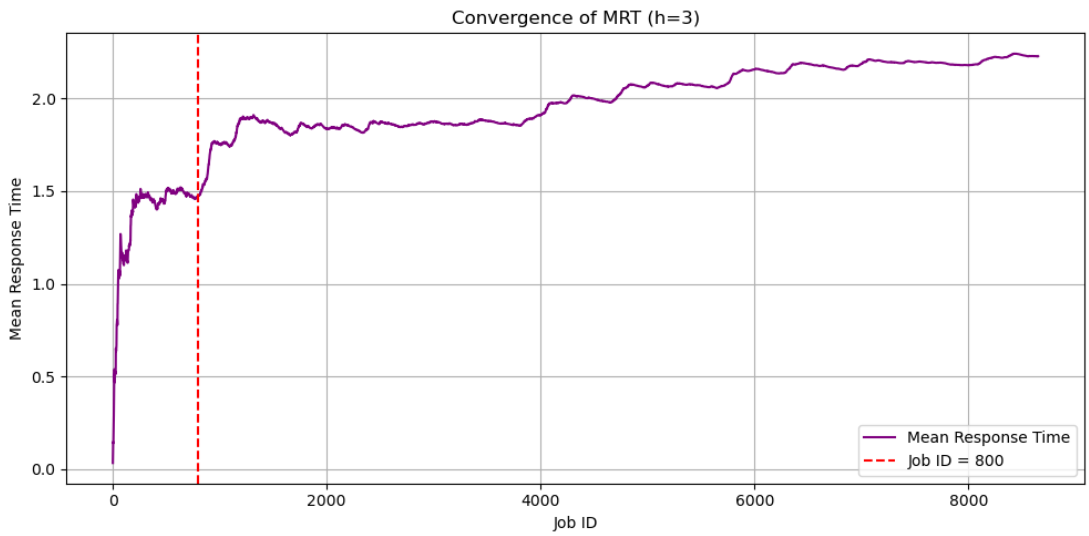
Therefore, we use W=600 as the benchmark because it is the smoothest. After Subjob Index 2000, the fluctuation starts to decrease, the curve becomes smoother, and the response time stabilises gradually.

Around Subjob Index 3500, the fluctuation is very small, the curve starts to flatten, and the response time stabilises at around 2.5 seconds.

Therefore, it is reasonable to choose Subjob Index 3500.



From the graph below, we can see that the fluctuation is very sharp at the beginning until it reaches a local minimum at 800. After 800, the curve rises slowly, but the trend has become milder and closer to stable.



Combining both observations, we can see that setting cutoff to 800 is a reasonable decision.

To analyze the simulation length, time_end_test.py runs 50 simulations for each time point by changing time_end, calculates the MRT average and 95% confidence interval, and smoothes the data using the moving average method. The result shows that the MRT is stabilized at 3.15-3.16 after time_end=8000,

which indicates that the system has entered a stable state. For the cutoff time, mrt_cutoff_time.py runs 5 simulations from 100 to 1200, and found that the MRT reaches a local minimum of 2.48 at cutoff=800, which indicates that the system is in stable state. In addition, slidingwindow.py analyzes the response time of 20 simulations using a sliding window and cumulative averaging, and finds that the curve flattens out after Subjob Index 3500 (about 833 seconds), further supporting the choice of cutoff=800.

Q3(b)

In Q3(b), we need to determine the optimal h value and evaluate the stability of the MRT. We used optimize_h.py to analyze the MRT for h=1 to 6and parewise.py to perform pairwise t-tests to compare the differences for h=2, 3, 4, 5 .

To assess the variation and stability of MRT at different threshold h values, I used 60 independent simulations for each h value. In each simulation, I fixed different random seeds to ensure the repeatability of the simulation results. For each h, I calculated the average MRT and 95% confidence interval for each of the 60 simulations.

I use a confidence interval plot to visualize the MRT and its uncertainty for different h. As can be seen from the plot, h = 3 performs best when it has the lowest MRT and the narrowest CI, showing that it is the best choice for this simulation.

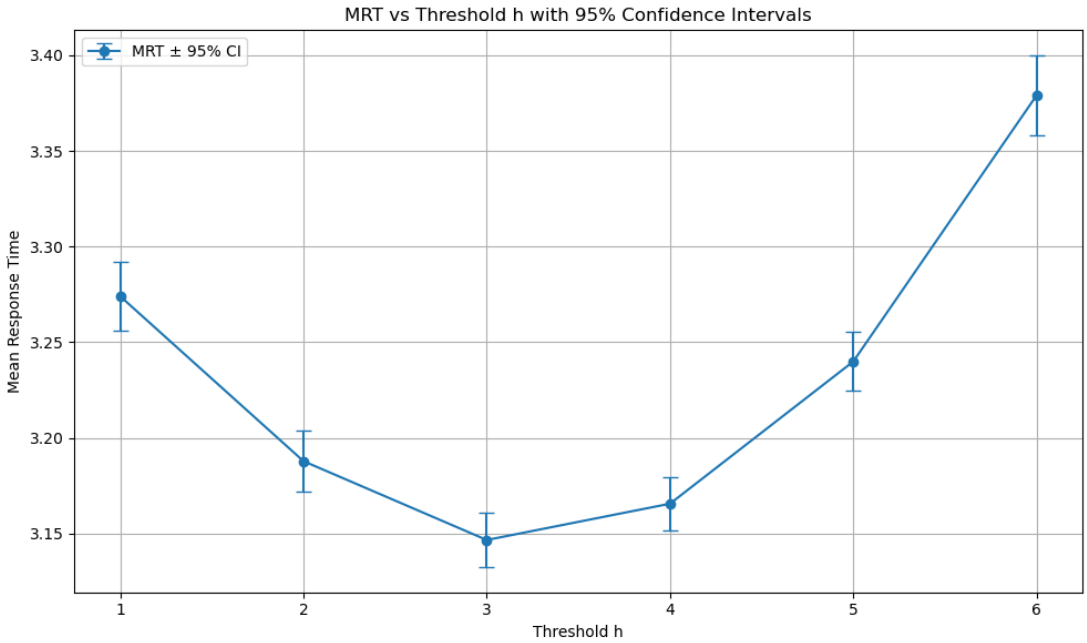
To assess the stability and statistical significance of the MRT at each h value, I used a 95% Confidence Interval to present the uncertainty.

The simulation results are tabulated below:

Threshold h	**Mean MRT**	**95% CI**
0	**3.3479**	**[3.2841, 3.4118]**
1	**3.2357**	**[3.1945, 3.2770]**
2	**3.1729**	**[3.1256, 3.2202]**
3	**3.1559**	**[3.1125, 3.1993]**
4	**3.2058**	**[3.1488, 3.2629]**
5	**3.2817**	**[3.2368, 3.3265]**
6	**3.3934**	**[3.3097, 3.4771]**

Graph analysis:

As can be seen from the graph below, the MRT is lowest at h=3, indicating that the setting has less variability and optimal performance.



In Figure “MRT vs Threshold h with 95% Confidence Intervals”, we have observed that there is an overlap between the 95% confidence intervals for Threshold h = 3 and h = 4. Specifically, the mean MRT for h = 3 is about 3.15 seconds and for h = 4 is about 3.18 seconds, but due to the overlapping CIs, we were unable to confirm the statistical significance of this difference at the 95% confidence level. This suggests that the difference in MRT between h = 3 and h = 4 may not be significant.

In order to find the best threshold h, we performed pairwise t-tests for h = 2, 3, 4, 5, with each h combination based on 10 independent simulation replications. The p-values and results of these tests are shown in the figure above and are judged at the significance level of $\alpha = 0.05$.

From the graph, all p-values are less than 0.05, which means that the MRT differences for each pair of h-values are statistically significant. Since all combinations have significant differences, we can further select the best h-value based on the mean MRT values observed in the simulation. According to the

results of the previous analysis, the MRT is lowest when h = 3, so we determine that the best threshold is h = 3.

```
Pairwise T-Test Results (α = 0.05):
| Comparison    | p-value     | Result                 |
|---------------|-------------|------------------------|
h=2 and h=3 | 0.0013700702 | Notable Difference
h=2 and h=4 | 0.0000010194 | Notable Difference
h=2 and h=5 | 0.0000034247 | Notable Difference
h=3 and h=4 | 0.0000148254 | Notable Difference
h=3 and h=5 | 0.0000425965 | Notable Difference
h=4 and h=5 | 0.0001005129 | Notable Difference
```

In optimize_h.py, we ran 60 independent simulations for each value of h, each time using different seeds to generate arrival times and service times, and calculated the mean MRT and 95% confidence intervals. The graph shows that h=3 has the lowest MRT (3.1559) and the narrowest confidence interval ([3.1125, 3.1993]), indicating the best performance and high stability. For further confirmation, parewise.py ran 10 simulations for each of h=2, 3, 4, and 5, calculated the MRT difference for each pair of h-values, and performed a t-test, with all p-values < 0.05 confirming a significant difference. Although the overlapping CIs suggested that the difference might not be significant, a subsequent pairwise t-test showed a p-value < 0.05 for h = 3 vs. h = 4, confirming that the difference was statistically significant and thus supporting h = 3 as the best choice.

Reference:

This assignment is referenced from https://github.com/co4524/M-M-m-SystemSimulation/blob/master/R07944048_hw2.py on M-m-m System Simulation by co4524 on Jun 7, 2019. python file.